Slip 1

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

```
#include <stdio.h>
int main() {
    int n, e, i, j, u, v, adj[20][20] = {0};
    printf("Vertices: "); scanf("%d", &n);
    printf("Edges: "); scanf("%d", &e);
    printf("Enter edges (u v):\n");
    for (i = 0; i < e; i++) { scanf("%d %d", &u, &v); adj[u][v] = adj[v][u] = 1; }
    printf("\nAdjacency Matrix:\n");
    for (i = 0; i < n; i++) { for (j = 0; j < n; j++) printf("%d ", adj[i][j]); printf("\n"); }
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Project(p_no, pname, ptype, duration); Employee(eno, ename, qualification, joining_date); Works_On(p_no, eno, start_date, no_of_hours_worked)

ER Diagram

Project **pno(PK), pname, ptype, duration**	*Works_On* *M:N*	**Employee** **eno(PK), ename, qualification, joining_date**

Constraints: duration > 0, pname NOT NULL. Descriptive attrs: start_date, no_of_hours_worked.

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_project_duration()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.duration <= 0 THEN RAISE EXCEPTION 'Duration must be greater than zero'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_duration
BEFORE INSERT ON Project
FOR EACH ROW EXECUTE FUNCTION check_project_duration();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE calc_three_nums(a INT, b INT, c INT)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Addition: %', a + b + c;
    RAISE NOTICE 'Subtraction: %', a - b - c;
    RAISE NOTICE 'Multiplication: %', a * b * c;
END;
$$;
```

Queries for creating database:

1) create table project (pno int primary key, pname char (30) not null, ptype char (20), duration integer check (duration > 0));
2) create table employee (eno integer primary key, ename char (20), qualification char (15), joining_date date);
3) create table project_employee (pno integer references project, eno integer references employee, start_date date, noofhours integer);

Slip 2

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to find the minimum and maximum values in a Binary Search Tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int findMin(struct Node* root) { while (root->left) root = root->left; return root->data; }
int findMax(struct Node* root) { while (root->right) root = root->right; return root->data; }
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Minimum: %d\n", findMin(root));
    printf("Maximum: %d\n", findMax(root));
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Person(p no, pname, birthdate, income); Area(aname, area_type); area_type = 'urban' or 'rural'.

ER Diagram

Area **aname(PK), area_type**	*lives_in* *1:M*	**Person** **pno(PK), pname, birthdate, income**

Q2.1 Solution (Stored Function - Option A)

```
CREATE OR REPLACE FUNCTION count_persons_in_area(area_name VARCHAR)
RETURNS INTEGER AS $$
DECLARE cnt INTEGER;
BEGIN
    SELECT COUNT(*) INTO cnt FROM Person WHERE aname = area_name;
    RETURN cnt;
END;
$$ LANGUAGE plpgsql;
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE divide_two_nums(a INT, b INT)
LANGUAGE plpgsql AS $$
BEGIN
    IF b = 0 THEN RAISE EXCEPTION 'Division by zero error'; END IF;
    RAISE NOTICE 'Division: %', a / b;
END;
$$;
```

Queries for creating database:

1) create table area (ano int primary key, aname varchar (20), area_type varchar (5) check (atype in ('Urban', 'Rural')));
2) create table person (pno integer primary key, pname varchar (20), birthdate date, income money, ano int references area);

Slip 3

Chosen: Write a C program to find the height of the tree and check whether given tree is balanced or not.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int height(struct Node* root) {
    if (root == NULL) return 0;
    int lh = height(root->left), rh = height(root->right);
    return (lh > rh ? lh : rh) + 1;
}
int isBalanced(struct Node* root) {
    if (root == NULL) return 1;
    int lh = height(root->left), rh = height(root->right);
    if (abs(lh-rh) <= 1 && isBalanced(root->left) && isBalanced(root->right)) return 1;
    return 0;
}
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Height: %d\n", height(root));
    printf(isBalanced(root) ? "Balanced\n" : "Not Balanced\n");
    return 0;
}
```

Schema: Student(s_no, s_name, address, class); Teacher(t_no, t_name, qualification, experience); Teaches(s_no, t_no, subject, marks_scored)

ER Diagram

Student **s_no(PK), s_name, class**	*Teaches* *M:N*	**Teacher** **t_no(PK), t_name, qualification, experience**

Constraints: s_name, t_name NOT NULL; marks_scored > 0.

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_student_number()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.s_no <= 0 THEN RAISE EXCEPTION 'Invalid Number'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_sno
BEFORE INSERT ON Student
FOR EACH ROW EXECUTE FUNCTION check_student_number();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE min_max_two(a INT, b INT)
LANGUAGE plpgsql AS $$
BEGIN
    IF a < b THEN RAISE NOTICE 'Min: %, Max: %', a, b;
    ELSE RAISE NOTICE 'Min: %, Max: %', b, a; END IF;
END;
$$;
```

1) create table student (sno integer primary key, sname char(20) not null, address char(25), class char(10));
2) create table teacher (tno integer primary key, tname char(10) not null, qualification char(10), experience integer);
3) create table student_teacher (sno integer references student, tno integer references teacher, subject varchar(10), marks_scored integer check (marks_scored > 0));

Slip 4

Chosen: Write a C program to find the height of the tree and check whether given tree is balanced or not.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int height(struct Node* root) {
    if (root == NULL) return 0;
    int lh = height(root->left), rh = height(root->right);
    return (lh > rh ? lh : rh) + 1;
}
int isBalanced(struct Node* root) {
    if (root == NULL) return 1;
    int lh = height(root->left), rh = height(root->right);
    if (abs(lh-rh) <= 1 && isBalanced(root->left) && isBalanced(root->right)) return 1;
    return 0;
}
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Height: %d\n", height(root));
    printf(isBalanced(root) ? "Balanced\n" : "Not Balanced\n");
    return 0;
}
```

ER Diagram

Actor **a_name(PK), role, charges**	*Acts_In* *M:N*	**Movies** **m_name(PK), release_year, budget**	*Produces* *M:N*	**Producer** **producer_id(PK), name**

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_movie_budget()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.budget < 5000000 THEN RAISE EXCEPTION 'Budget should be minimum 50 lakh'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_budget
BEFORE INSERT ON Movies
FOR EACH ROW EXECUTE FUNCTION check_movie_budget();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE check_number(num INT)
LANGUAGE plpgsql AS $$
BEGIN
    IF num > 0 THEN RAISE NOTICE 'Positive';
    ELSIF num < 0 THEN RAISE NOTICE 'Negative';
    ELSE RAISE NOTICE 'Zero'; END IF;
END;
$$;
```

1) create table movies (mno int primary key, mname varchar (25), releaseyear integer, budget money);
2) create table actor (ano int primary key, aname char (30), role char (30), charges money, aaddress varchar(30));
3) create table producer (pid integer primary key, pname char (30), paddress varchar (30));
4) create table movie_actor(mno int references movies, ano int references actor);
5) create table movie_producer(mno int references movies, pno int references producer);

Slip 8

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

```
#include <stdio.h>
int main() {
    int n, e, i, j, u, v, adj[20][20] = {0};
    printf("Vertices: "); scanf("%d", &n);
    printf("Edges: "); scanf("%d", &e);
    printf("Enter edges (u v):\n");
    for (i = 0; i < e; i++) { scanf("%d %d", &u, &v); adj[u][v] = adj[v][u] = 1; }
    printf("\nAdjacency Matrix:\n");
    for (i = 0; i < n; i++) { for (j = 0; j < n; j++) printf("%d ", adj[i][j]); printf("\n"); }
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Train(train_no, train_name, source_stn, dest_stn, ...); Passenger(passenger_id, passenger_name, age, gender); Ticket(train_no, passenger_id, ticket_no, bogie_no, no_of_berths, tdate, ticket_amt, status)

ER Diagram

Train **train_no(PK), train_name, source_stn, dest_stn**	*Ticket* *M:N*	**Passenger** **passenger_id(PK), passenger_name, age, gender**

Descriptive attributes: ticket_no, bogie_no, no_of_berths, tdate, ticket_amt, status ('W'/'C').

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_passenger_age()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.age > 5 THEN RAISE NOTICE 'Age above 5 will be charged full fare'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_passenger_age
AFTER INSERT ON Passenger
FOR EACH ROW EXECUTE FUNCTION check_passenger_age();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE calc_two_nums(a INT, b INT)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Addition: %', a + b;
    RAISE NOTICE 'Subtraction: %', a - b;
    RAISE NOTICE 'Multiplication: %', a * b;
END;
$$;
```

Queries for creating database:

1) Create table train (trno int primary key, trname varchar(20), depart_time time , arrival_time time, source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int) ;
2) Create table passenger (pid int primary key, pname varchar(20), address varchar(30), age int ,gender char);
3) Create table ticket (trno int references train, pid int references passenger, ticket_no int ,bogie_no int, no_of_berths int ,tdate date , ticket_amt decimal(7,2),status char check(status in ('W', 'C')));

Slip 9

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to find the height of the tree and check whether given tree is balanced or not.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int height(struct Node* root) {
    if (root == NULL) return 0;
    int lh = height(root->left), rh = height(root->right);
    return (lh > rh ? lh : rh) + 1;
}
int isBalanced(struct Node* root) {
    if (root == NULL) return 1;
    int lh = height(root->left), rh = height(root->right);
    if (abs(lh-rh) <= 1 && isBalanced(root->left) && isBalanced(root->right)) return 1;
    return 0;
}
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Height: %d\n", height(root));
    printf(isBalanced(root) ? "Balanced\n" : "Not Balanced\n");
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Student(sreg_no, name, class); Competition(c_no, name, C_type); Participates(sreg_no, c_no, rank, year, prize)

ER Diagram

Student **sreg_no(PK), name, class**	*Participates* *M:N*	**Competition** **c_no(PK), name, C_type**

Q2.1 Solution (Function with Cursor - Option B)

```
CREATE OR REPLACE FUNCTION count_prizes_2020(stud_name VARCHAR)
RETURNS INTEGER AS $$
DECLARE cnt INTEGER := 0; rec RECORD;
    cur CURSOR FOR SELECT p.prize FROM Participates p
        JOIN Student s ON p.sreg_no = s.sreg_no
        WHERE s.name = stud_name AND p.year = 2020;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        cnt := cnt + rec.prize;
    END LOOP;
    CLOSE cur;
    RETURN cnt;
END;
$$ LANGUAGE plpgsql;
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE divide_two_nums(a INT, b INT)
LANGUAGE plpgsql AS $$
BEGIN
    IF b = 0 THEN RAISE EXCEPTION 'Division by zero error'; END IF;
    RAISE NOTICE 'Division: %', a / b;
END;
$$;
```

1) Create table student (sreg_no int primary key, name char(30), class char(10));
2) Create table competition (c_no int primary key, name char(20), C_type char(15));
3) Create table student_competition (sreg_no int references student, c_no int references competition, rank int, year int, prize int);

Slip 10

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to find the minimum and maximum values in a Binary Search Tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int findMin(struct Node* root) { while (root->left) root = root->left; return root->data; }
int findMax(struct Node* root) { while (root->right) root = root->right; return root->data; }
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Minimum: %d\n", findMin(root));
    printf("Maximum: %d\n", findMax(root));
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Book(b_no, b_name, pub_name, b_price); Author(a_no, a_name, qualification, address); Writes(b_no, a_no)

ER Diagram

Book **b_no(PK), b_name, pub_name, b_price**	*Writes* *M:N*	**Author** **a_no(PK), a_name, qualification**

Constraint: pub_name NOT NULL.

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_book_price()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.b_price > 1000 THEN RAISE NOTICE 'Price is so high'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_book_price
AFTER INSERT ON Book
FOR EACH ROW EXECUTE FUNCTION check_book_price();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE print_even_numbers()
LANGUAGE plpgsql AS $$
DECLARE i INT;
BEGIN
    FOR i IN 1..50 LOOP IF i % 2 = 0 THEN RAISE NOTICE '%', i; END IF; END LOOP;
END;
$$;
```

Queries for creating database:

1) Create table book(b_no int primary key, b_name varchar (20), pub_name varchar (10) not null, b_price float);
2) Create table author (a_no int primary key, a_name varchar (20), qualification varchar (15), address varchar (15));
3) Create table book_author(b_no int refernces book, a_no int references author);

Slip 11

Section 1: Data Structures-II (15 Marks)

Chosen: Write a program which uses binary search tree library and counts the total nodes and total leaf nodes in the tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int count(struct Node* root) {
    if (root == NULL) return 0;
    return 1 + count(root->left) + count(root->right);
}
int countLeaf(struct Node* root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeaf(root->left) + countLeaf(root->right);
}
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Total Nodes: %d\n", count(root));
    printf("Total Leaf Nodes: %d\n", countLeaf(root));
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Student(rollno, s_name, class); Subject(scode, subject_name); Enrolled(rollno, scode, marks)

ER Diagram

Student **rollno(PK), s_name, class**	*Enrolled* *M:N*	**Subject** **scode(PK), subject_name**

Descriptive attribute: marks.

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION notify_student_delete()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'student record is being deleted';
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_student_delete
BEFORE DELETE ON Student
FOR EACH ROW EXECUTE FUNCTION notify_student_delete();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE sum_avg_n(n INT)
LANGUAGE plpgsql AS $$
DECLARE sum_val INT := 0; i INT;
BEGIN
    FOR i IN 1..n LOOP sum_val := sum_val + i; END LOOP;
    RAISE NOTICE 'Sum: %', sum_val;
    RAISE NOTICE 'Average: %', sum_val::FLOAT / n;
END;
$$;
```

1) Create table student (rollno int primary key, s_name varchar(20), class varchar(15));
2) Create table subject (scode varchar(10), subject_name varchar(20));
3) Create table student_subject (rollno int references student, scode varchar(10) references subject, marks integer);

Slip 15

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to implement a hash table using open addressing with linear probing. Perform Insert, Search and Display.

```
#include <stdio.h>
#define SIZE 10
int hash[SIZE];
void init() { int i; for (i = 0; i < SIZE; i++) hash[i] = -1; }
int hashFunc(int key) { return key % SIZE; }
void insert(int key) {
    int index = hashFunc(key), i;
    for (i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;
        if (hash[pos] == -1 || hash[pos] == -2) { hash[pos] = key; return; }
    }
    printf("Table full\n");
}
void search(int key) {
    int index = hashFunc(key), i;
    for (i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;
        if (hash[pos] == key) { printf("Found at %d\n", pos); return; }
        if (hash[pos] == -1) break;
    }
    printf("Not found\n");
}
void display() { int i; for (i = 0; i < SIZE; i++) printf("[%d]: %d\n", i, hash[i]); }
int main() {
    int choice, key;
    init();
    do {
        printf("1.Insert 2.Search 3.Display 4.Exit\nChoice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: printf("Key: "); scanf("%d", &key); insert(key); break;
            case 2: printf("Key: "); scanf("%d", &key); search(key); break;
            case 3: display(); break;
        }
    } while (choice != 4);
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Project(pno, p_name, ptype, duration); Employee(eno, e_name, qualification, joindate); Works_On(pno, eno, start_date, no_of_hours_worked)

ER Diagram

Project **pno(PK), p_name, ptype, duration**	*Works_On* *M:N*	**Employee** **eno(PK), e_name, qualification, joindate**

Q2.1 Solution (Trigger - Option A)

```
CREATE OR REPLACE FUNCTION check_project_duration()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.duration <= 0 THEN RAISE EXCEPTION 'Duration must be greater than zero'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_project_duration
BEFORE INSERT ON Project
FOR EACH ROW EXECUTE FUNCTION check_project_duration();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE insert_project(
    p_no INT, p_name CHAR(30), p_type CHAR(20), p_dur INT)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO Project(pno, p_name, ptype, duration) VALUES (p_no, p_name, p_type, p_dur);
    RAISE NOTICE 'Project inserted successfully';
END;
$$;
```

Slip 17

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to find the minimum and maximum values in a Binary Search Tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int findMin(struct Node* root) { while (root->left) root = root->left; return root->data; }
int findMax(struct Node* root) { while (root->right) root = root->right; return root->data; }
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Minimum: %d\n", findMin(root));
    printf("Maximum: %d\n", findMax(root));
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Student(sreg_no, name, class); Competition(c_no, name, C_type); Participates(sreg_no, c_no, rank, year, prize)

ER Diagram

Student **sreg_no(PK), name, class**	*Participates* *M:N*	**Competition** **c_no(PK), name, C_type**

Q2.1 Solution (Trigger - Option A)

```
CREATE OR REPLACE FUNCTION notify_competition_update()
RETURNS TRIGGER AS $$
BEGIN
    RAISE NOTICE 'competition record is being updated';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_competition_update
BEFORE UPDATE ON Competition
FOR EACH ROW EXECUTE FUNCTION notify_competition_update();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE sum_first_50()
LANGUAGE plpgsql AS $$
DECLARE sum_val INT := 0; i INT;
BEGIN
    FOR i IN 1..50 LOOP sum_val := sum_val + i; END LOOP;
    RAISE NOTICE 'Sum of first 50 numbers: %', sum_val;
END;
$$;
```

Slip 18

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program that accepts the vertices and edges of a graph and stores it as an adjacency matrix. Display the adjacency matrix.

```
#include <stdio.h>
int main() {
    int n, e, i, j, u, v, adj[20][20] = {0};
    printf("Vertices: "); scanf("%d", &n);
    printf("Edges: "); scanf("%d", &e);
    printf("Enter edges (u v):\n");
    for (i = 0; i < e; i++) { scanf("%d %d", &u, &v); adj[u][v] = adj[v][u] = 1; }
    printf("\nAdjacency Matrix:\n");
    for (i = 0; i < n; i++) { for (j = 0; j < n; j++) printf("%d ", adj[i][j]); printf("\n"); }
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Item(itemno, itemname); Supplier(supplier_no, supplier_name, city); Supplies(itemno, supplier_no, rate, quantity)

ER Diagram

Item **itemno(PK), itemname**	*Supplies* *M:N*	**Supplier** **supplier_no(PK), supplier_name, city**

Descriptive attributes: rate, quantity.

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_rate_value()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.rate < 50 THEN RAISE EXCEPTION 'Rate cannot be less than 50'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_rate
BEFORE INSERT OR UPDATE ON Supplies
FOR EACH ROW EXECUTE FUNCTION check_rate_value();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE max_of_three(a INT, b INT, c INT)
LANGUAGE plpgsql AS $$
DECLARE max_val INT;
BEGIN
    max_val := a;
    IF b > max_val THEN max_val := b; END IF;
    IF c > max_val THEN max_val := c; END IF;
    RAISE NOTICE 'Maximum: %', max_val;
END;
$$;
```

Slip 21

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to implement a hash table using open addressing with linear probing. Perform Insert, Search and Display.

```
#include <stdio.h>
#define SIZE 10
int hash[SIZE];
void init() { int i; for (i = 0; i < SIZE; i++) hash[i] = -1; }
int hashFunc(int key) { return key % SIZE; }
void insert(int key) {
    int index = hashFunc(key), i;
    for (i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;
        if (hash[pos] == -1 || hash[pos] == -2) { hash[pos] = key; return; }
    }
    printf("Table full\n");
}
void search(int key) {
    int index = hashFunc(key), i;
    for (i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;
        if (hash[pos] == key) { printf("Found at %d\n", pos); return; }
        if (hash[pos] == -1) break;
    }
    printf("Not found\n");
}
void display() { int i; for (i = 0; i < SIZE; i++) printf("[%d]: %d\n", i, hash[i]); }
int main() {
    int choice, key;
    init();
    do {
        printf("1.Insert 2.Search 3.Display 4.Exit\nChoice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: printf("Key: "); scanf("%d", &key); insert(key); break;
            case 2: printf("Key: "); scanf("%d", &key); search(key); break;
            case 3: display(); break;
        }
    } while (choice != 4);
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Doctor(d_no, d_name, specialization, phone_no, charges); Hospital(h_no, h_name, city); Works_At(d_no, h_no)

ER Diagram

Hospital **h_no(PK), h_name, city**	*Works_At* *1:M*	**Doctor** **d_no(PK), d_name, specialization, charges**

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_doctor_charges()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.charges < 0 THEN RAISE EXCEPTION 'Charges cannot be negative'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_doctor_charges
BEFORE INSERT OR UPDATE ON Doctor
FOR EACH ROW EXECUTE FUNCTION check_doctor_charges();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE insert_doctor(
    d_no INT, d_name VARCHAR(30), spec VARCHAR(35), phone INT, charges INT)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO Doctor(d_no, d_name, specialization, phone_no, charges)
    VALUES (d_no, d_name, spec, phone, charges);
    RAISE NOTICE 'Doctor inserted successfully';
END;
$$;
```

Slip 22

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program that accepts the vertices and edges of a graph and store it as an adjacency list. Implement function to traverse the graph using Depth First Search (DFS) traversal.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int v; struct Node* next; };
struct Node* adj[20]; int visited[20];
struct Node* createNode(int v) {
    struct Node* n = malloc(sizeof(struct Node));
    n->v = v; n->next = NULL; return n;
}
void addEdge(int u, int v) {
    struct Node* n = createNode(v); n->next = adj[u]; adj[u] = n;
    n = createNode(u); n->next = adj[v]; adj[v] = n;
}
void DFS(int v) {
    struct Node* temp;
    visited[v] = 1; printf("%d ", v);
    for (temp = adj[v]; temp; temp = temp->next)
        if (!visited[temp->v]) DFS(temp->v);
}
int main() {
    int n, e, i, u, v, start;
    printf("Vertices Edges: "); scanf("%d %d", &n, &e);
    for (i = 0; i < n; i++) { adj[i] = NULL; visited[i] = 0; }
    for (i = 0; i < e; i++) { scanf("%d %d", &u, &v); addEdge(u, v); }
    printf("Start: "); scanf("%d", &start);
    printf("DFS: "); DFS(start); printf("\n");
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Bus(bus_no, capacity, depot_name); Route(route_no, source, destination, no_of_stations); Driver(driver_no, driver_name, license_no, address, d_age, salary); Bus_Route(bus_no, route_no); Allotment(bus_no, driver_no, duty_date, shift)

ER Diagram

Bus **bus_no(PK), capacity, depot_name**	*runs_on* *M:1*	**Route** **route_no(PK), source, destination**

Bus **bus_no(PK), capacity**	*Allotment* *M:N*	**Driver** **driver_no(PK), driver_name, license_no(UQ), d_age**

Constraints: license_no unique, capacity NOT NULL. Descriptive: duty_date, shift (1/2).

Q2.1 Solution (Trigger - Option A)

```
CREATE OR REPLACE FUNCTION check_driver_age()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.d_age < 18 OR NEW.d_age > 50 THEN RAISE EXCEPTION 'invalid entry'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_driver_age
BEFORE INSERT ON Driver
FOR EACH ROW EXECUTE FUNCTION check_driver_age();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE print_odd_numbers(n INT)
LANGUAGE plpgsql AS $$
DECLARE i INT;
BEGIN
    FOR i IN 1..n LOOP IF i % 2 != 0 THEN RAISE NOTICE '%', i; END IF; END LOOP;
END;
$$;
```

Slip 23

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to find the height of the tree and check whether given tree is balanced or not.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int height(struct Node* root) {
    if (root == NULL) return 0;
    int lh = height(root->left), rh = height(root->right);
    return (lh > rh ? lh : rh) + 1;
}
int isBalanced(struct Node* root) {
    if (root == NULL) return 1;
    int lh = height(root->left), rh = height(root->right);
    if (abs(lh-rh) <= 1 && isBalanced(root->left) && isBalanced(root->right)) return 1;
    return 0;
}
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Height: %d\n", height(root));
    printf(isBalanced(root) ? "Balanced\n" : "Not Balanced\n");
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Car(c_no, owner, model, color); Driver(driver_no, driver_name, license_no, d_age, salary); Drives(c_no, driver_no)

ER Diagram

Car **c_no(PK), owner, model, color**	*Drives* *M:N*	**Driver** **driver_no(PK), driver_name, license_no, d_age**

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_driver_age()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.d_age < 21 THEN RAISE EXCEPTION 'Driver age must be at least 21'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_driver_age
BEFORE INSERT OR UPDATE ON Driver
FOR EACH ROW EXECUTE FUNCTION check_driver_age();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE sum_first_100()
LANGUAGE plpgsql AS $$
DECLARE sum_val INT := 0; i INT;
BEGIN
    FOR i IN 1..100 LOOP sum_val := sum_val + i; END LOOP;
    RAISE NOTICE 'Sum of first 100 numbers: %', sum_val;
END;
$$;
```

Slip 25

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program for the implementation of Dijkstra's shortest path algorithm for finding shortest path from a given source vertex using adjacency cost matrix.

```
#include <stdio.h>
#define INF 9999
#define MAX 20
void dijkstra(int cost[MAX][MAX], int n, int src) {
    int dist[MAX], visited[MAX] = {0}, i, j, min, u;
    for (i = 0; i < n; i++) dist[i] = cost[src][i];
    dist[src] = 0; visited[src] = 1;
    for (i = 1; i < n; i++) {
        min = INF;
        for (j = 0; j < n; j++)
            if (!visited[j] && dist[j] < min) { min = dist[j]; u = j; }
        visited[u] = 1;
        for (j = 0; j < n; j++)
            if (!visited[j] && cost[u][j] != INF && dist[u] + cost[u][j] < dist[j])
                dist[j] = dist[u] + cost[u][j];
    }
    printf("Vertex\tDistance\n");
    for (i = 0; i < n; i++) printf("%d\t%d\n", i, dist[i]);
}
int main() {
    int n, i, j, src, cost[MAX][MAX];
    printf("Vertices: "); scanf("%d", &n);
    printf("Cost matrix (9999 = no edge):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) scanf("%d", &cost[i][j]);
    printf("Source: "); scanf("%d", &src);
    dijkstra(cost, n, src);
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Customer(cno, cname, city); Account(a_no, a_type, opening_date, balance); Holds(cno, a_no)

ER Diagram

Customer **cno(PK), cname, city**	*Holds* *1:M*	**Account** **a_no(PK), a_type, opening_date, balance**

Q2.1 Solution (Cursor Function - Option A)

```
CREATE OR REPLACE FUNCTION show_customers_by_city(city_name VARCHAR)
RETURNS VOID AS $$
DECLARE rec RECORD;
    cur CURSOR FOR
        SELECT c.cno, c.cname, c.city, a.a_no, a.a_type, a.balance
        FROM Customer c
        JOIN Holds h ON c.cno = h.cno
        JOIN Account a ON h.a_no = a.a_no
        WHERE c.city = city_name;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'CNo: %, Name: %, City: %, ANo: %, Type: %, Balance: %',
            rec.cno, rec.cname, rec.city, rec.a_no, rec.a_type, rec.balance;
    END LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE show_pune_customers()
LANGUAGE plpgsql AS $$
DECLARE rec RECORD;
BEGIN
    FOR rec IN SELECT * FROM Customer WHERE city = 'Pune' LOOP
        RAISE NOTICE 'CNo: %, Name: %, City: %', rec.cno, rec.cname, rec.city;
    END LOOP;
END;
$$;
```

Slip 26

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to find the minimum and maximum values in a Binary Search Tree.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node* create(int val) {
    struct Node* n = malloc(sizeof(struct Node));
    n->data = val; n->left = n->right = NULL; return n;
}
struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return create(val);
    if (val < root->data) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
int findMin(struct Node* root) { while (root->left) root = root->left; return root->data; }
int findMax(struct Node* root) { while (root->right) root = root->right; return root->data; }
int main() {
    struct Node* root = NULL;
    int n, i, val;
    printf("Nodes: "); scanf("%d", &n);
    for (i = 0; i < n; i++) { scanf("%d", &val); root = insert(root, val); }
    printf("Minimum: %d\n", findMin(root));
    printf("Maximum: %d\n", findMax(root));
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Item(itemno, Itemname, quantity); Supplier(supplierno, Suppliername, city); Supplies(itemno, supplierno, rate)

ER Diagram

Item **itemno(PK), Itemname, quantity**	*Supplies* *M:N*	**Supplier** **supplierno(PK), Suppliername, city**

Descriptive attribute: rate.

Q2.1 Solution (Cursor Function - Option A)

```
CREATE OR REPLACE FUNCTION show_item_suppliers(item_name VARCHAR)
RETURNS VOID AS $$
DECLARE rec RECORD;
    cur CURSOR FOR
        SELECT i.Itemname, s.Suppliername, su.rate
        FROM Item i
        JOIN Supplies su ON i.itemno = su.itemno
        JOIN Supplier s ON su.supplierno = s.supplierno
        WHERE i.Itemname = item_name;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Item: %, Supplier: %, Rate: %',
            rec.Itemname, rec.Suppliername, rec.rate;
    END LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE calc_all_ops(a INT, b INT)
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Addition: %', a + b;
    RAISE NOTICE 'Subtraction: %', a - b;
    RAISE NOTICE 'Multiplication: %', a * b;
    IF b = 0 THEN RAISE EXCEPTION 'Division by zero error';
    ELSE RAISE NOTICE 'Division: %', a::FLOAT / b; END IF;
END;
$$;
```

Slip 29

Section 1: Data Structures-II (15 Marks)

Chosen: Write a C program to implement a hash table using open addressing with linear probing. Perform Insert, Search and Display.

```
#include <stdio.h>
#define SIZE 10
int hash[SIZE];
void init() { int i; for (i = 0; i < SIZE; i++) hash[i] = -1; }
int hashFunc(int key) { return key % SIZE; }
void insert(int key) {
    int index = hashFunc(key), i;
    for (i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;
        if (hash[pos] == -1 || hash[pos] == -2) { hash[pos] = key; return; }
    }
    printf("Table full\n");
}
void search(int key) {
    int index = hashFunc(key), i;
    for (i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;
        if (hash[pos] == key) { printf("Found at %d\n", pos); return; }
        if (hash[pos] == -1) break;
    }
    printf("Not found\n");
}
void display() { int i; for (i = 0; i < SIZE; i++) printf("[%d]: %d\n", i, hash[i]); }
int main() {
    int choice, key;
    init();
    do {
        printf("1.Insert 2.Search 3.Display 4.Exit\nChoice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: printf("Key: "); scanf("%d", &key); insert(key); break;
            case 2: printf("Key: "); scanf("%d", &key); search(key); break;
            case 3: display(); break;
        }
    } while (choice != 4);
    return 0;
}
```

Section 2: Database Management Systems-II (15 Marks)

Schema: Student(Roll_No, Sname, Sclass); Teacher(T_No, Tname, Experience); Teaches(Roll_No, T_No, Subject)

ER Diagram

Student **Roll_No(PK), Sname, Sclass**	*Teaches* *M:N*	**Teacher** **T_No(PK), Tname, Experience**

Descriptive attribute: Subject.

Q2.1 Solution (Trigger - Option B)

```
CREATE OR REPLACE FUNCTION check_roll_number()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Roll_No <= 0 THEN RAISE EXCEPTION 'Invalid Roll Number'; END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_roll_number
BEFORE INSERT ON Student
FOR EACH ROW EXECUTE FUNCTION check_roll_number();
```

Q2.2 Solution (Procedure)

```
CREATE OR REPLACE PROCEDURE sum_avg_n(n INT)
LANGUAGE plpgsql AS $$
DECLARE sum_val INT := 0; i INT;
BEGIN
    FOR i IN 1..n LOOP sum_val := sum_val + i; END LOOP;
    RAISE NOTICE 'Sum: %', sum_val;
    RAISE NOTICE 'Average: %', sum_val::FLOAT / n;
END;
$$;
```